

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ 5.1

Тема: *Обобщение, недообучение и переобучение. Анализ кривых обучения.*

1. Цели практического занятия

1. Научиться различать недообучение и переобучение.
2. Освоить анализ графиков функции потерь (train/val loss).
3. Построить простую нейронную сеть и зафиксировать динамику её обучения.
4. Научиться управлять переобучением через параметры модели и гиперпараметры.
5. Провести исследовательский эксперимент: как меняются кривые обучения при
 - разных размерах модели,
 - разном количестве эпох,
 - разных параметрах регуляризации,
 - разном размере обучающей выборки.

2. Теоретическое введение

В соответствии с Лекцией 5, способность модели к обобщению является ключевой характеристикой любой системы машинного обучения. При построении модели важно понимать, что её задача — не запомнить обучающие примеры, а вывести закономерности, применимые к новым данным. В этом практическом занятии мы сосредоточимся на том, как правильно диагностировать качество обучения модели, наблюдая за динамикой train loss и val loss.

Традиционно процесс обучения идёт по следующему сценарию:

- train loss уменьшается почти всегда монотонно;
- val loss должна уменьшаться, но после определённого момента может начать расти;
- увеличение разницы между train loss и val loss говорит о начале переобучения.

Формально эмпирический риск:

$$R_{\text{emp}}(\theta) = \frac{1}{N} \sum_{i=1}^N L(f_{\theta}(x_i), y_i)$$

всегда уменьшается при увеличении числа эпох, так как модель эффективно запоминает обучающие примеры.

Но истинный риск:

$$R_{\text{true}}(\theta) = \mathbb{E}_{(x,y) \sim p_{\text{test}}} [L(f_{\theta}(x), y)]$$

может расти.

Поэтому критически важно уметь анализировать рост ошибки на validation-выборке.

В рамках первой практики вы построите несколько моделей, исследуете, как изменяется обобщающая способность при увеличении числа эпох, и увидите на практике явление переобучения.

3. Порядок выполнения работы

Шаг 1. Подготовка данных

Мы будем использовать датасет Fashion-MNIST или MNIST (в ноутбуке используется MNIST для наглядности).

Студент должен выполнить:

```
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

Задачи:

- нормировать данные;
- привести изображения к виду (28, 28, 1);
- выполнить one-hot-кодирование меток.

Шаг 2. Построение простой модели

Студенты строят маленькую модель:

- Flatten
- Dense(64, relu)
- Dense(10, softmax)

Обучают 5–7 эпох.

Задачи:

- записать train loss
- записать val loss
- построить график

Ожидаемый результат:

- train loss $\uparrow\downarrow$ монотонно
- val loss $\downarrow$, затем начинает колебаться

Шаг 3. Создание модели большей емкости

Студенты добавляют слои:

- Dense(256, relu)
- Dense(128, relu)
- Dense(64, relu)

Требования:

- обучить до 20–25 эпох
- сохранить историю обучения
- построить графики

Ожидаемый результат:

- явное переобучение
- сильный разрыв train/val loss

Шаг 4. Управление переобучением

Студенты применяют:

Dropout

L2-регуляризацию

уменьшение числа параметров

И сравнивают графики.

4. Исследовательская часть

Студент самостоятельно исследует влияние следующих факторов:

1. Увеличение числа эпох $\rightarrow$ как меняется val loss?
2. Изменение размера батча (32 / 64 / 128).
3. Увеличение емкости модели (в 2 раза больше нейронов).
4. Уменьшение емкости модели (в 2 раза меньше нейронов).
5. Добавление L2-регуляризации.
6. Добавление Dropout (0.25 / 0.5).
7. Использование ReLU vs LeakyReLU.

8. Уменьшение обучающей выборки до 10%.
9. Использование EarlyStopping.
10. Сравнение двух графиков: *обучение без регуляризации и с регуляризацией.*

Для каждого исследования студент отвечает на вопросы:

- Как изменились графики train/val loss?
- Есть ли переобучение?
- Улучшилось ли обобщение?
- Какая модель оказалась лучше и почему?

5. Контрольные вопросы

1. Что такое обобщение?
2. Как выглядит график переобучения?
3. Что означает рост val loss при падении train loss?
4. Почему модель может недообучаться?
5. Каким образом увеличение глубины модели влияет на переобучение?
6. Что такое емкость модели?
7. Как уменьшить переобучение без изменения архитектуры?
8. Что делает Dropout?
9. Как действует L2-регуляризация?
10. Почему нужно следить за разницей train/val?
11. Что произойдет при увеличении batch size?
12. Зачем использовать проверочную выборку?

Практическое занятие 5.1

Обобщение, недообучение и переобучение

Полный ноутбук с примерами, комментариями и заданиями.

Загрузка библиотек

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from tensorflow.keras.datasets import mnist
```

```
from tensorflow.keras.utils import to_categorical
```

```
from tensorflow.keras import models, layers, regularizers
```

```
from tensorflow.keras.callbacks import EarlyStopping
```

1. Загрузка и подготовка данных

Загружаем данные MNIST

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

Нормализуем

```
x_train = x_train.astype("float32")/255.
```

```
x_test = x_test.astype("float32")/255.
```

Меняем форму

```
x_train = x_train.reshape((-1, 28, 28, 1))
```

```
x_test = x_test.reshape((-1, 28, 28, 1))
```

One-hot метки

```
y_train = to_categorical(y_train)
```

```
y_test = to_categorical(y_test)
```

```
x_train.shape, y_train.shape
```

```
## 2. Простая модель (малой емкости)
```

Посмотрим, возникает ли недообучение.

```
model_small = models.Sequential([  
    layers.Flatten(input_shape=(28,28,1)),  
    layers.Dense(64, activation='relu'),  
    layers.Dense(10, activation='softmax')  
])
```

```
model_small.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

```
history_small = model_small.fit(  
    x_train, y_train,  
    epochs=7,  
    batch_size=64,  
    validation_split=0.2,  
    verbose=1  
)
```

```
### Графики обучения простой модели
```

```
plt.plot(history_small.history['loss'], label='train loss')  
plt.plot(history_small.history['val_loss'], label='val loss')  
plt.legend(); plt.title("Small model: loss"); plt.show()
```

```
## 3. Модель большей емкости — ожидаем переобучение
```

```
model_big = models.Sequential([
```

```

layers.Flatten(input_shape=(28,28,1)),
layers.Dense(256, activation='relu'),
layers.Dense(128, activation='relu'),
layers.Dense(64, activation='relu'),
layers.Dense(10, activation='softmax')
])

model_big.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

history_big = model_big.fit(
    x_train, y_train,
    epochs=25,
    batch_size=64,
    validation_split=0.2,
    verbose=1
)

```

Графики обучения большой модели

```

plt.plot(history_big.history['loss'], label='train loss')
plt.plot(history_big.history['val_loss'], label='val loss')
plt.legend(); plt.title("Big model: loss"); plt.show()

```

4. Регуляризация (Dropout + L2)

```

model_reg = models.Sequential([
    layers.Flatten(input_shape=(28,28,1)),
    layers.Dense(256, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
    layers.Dropout(0.5),

```

```
layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
layers.Dropout(0.3),
layers.Dense(10, activation='softmax')
])

model_reg.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

history_reg = model_reg.fit(
    x_train, y_train,
    epochs=25,
    batch_size=64,
    validation_split=0.2,
    verbose=1
)
```

Графики обучения регуляризованной модели

```
plt.plot(history_reg.history['loss'], label='train loss')
plt.plot(history_reg.history['val_loss'], label='val loss')
plt.legend(); plt.title("Regularized model: loss"); plt.show()
```

Исследовательские задания

1. Измените batch_size на 32, 128. Как это повлияло на val_loss?
2. Ограничьте обучающую выборку 10% данных. Что произошло с переобучением?
3. Измените архитектуру: уберите один скрытый слой. Изменилась ли динамика обучения?
4. Добавьте EarlyStopping и сравните минимальные значения val_loss.
5. Увеличьте dropout до 0.7. Улучшилась ли общность?

6. Попробуйте другую функцию активации — LeakyReLU.
7. Посмотрите, что будет, если обучать маленькую модель 30 эпох.
8. Проанализируйте разницу в `train_accuracy` и `val_accuracy`.

Контрольные вопросы

1. Что показывает расхождение `train` и `val loss`?
2. Почему большая модель легко переобучается?
3. Как влияет количество эпох на обобщение?
4. Зачем нужен `dropout`?
5. Как действует L2-регуляризация?
6. Почему важно использовать `validation_split`?
7. Что произойдёт при слишком маленьком `batch size`?
8. Как отличить недообучение от переобучения по графику?